



GUJARAT TECHNOLOGICAL UNIVERSITY
Shree Swami Atmanand Saraswati Institute of Technology

A
Micro Project Report on
Movie / Netflix Dataset Analyzer

Under subject of
Python for Data Science (BE05000231)

B.E 3rd Year, Semester – V
COMPUTER ENGINEERING
Under the Guidance of
Prof. Ami D. Patel

Submitted by group:

Sr. No.	Name of Student	Enrollment No.
1	Hani Dharsandiya	240760107042
2	Khushali Ghodadra	240760107052
3	Drashti Lakhani	240760107043
4	Krupa Baldha	240760107009
5	Vinanti Gopani	240760107058
6	Hisha Anghan	240760107005

Academic year: 2026-2027

SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF TECHNOLOGY

Opp. Kapodra Police Station, Varachha Road, Surat – 395006



CERTIFICATE

This is to certify that the Project entitled **Movie / Netflix Dataset Analyzer** has been prepared by **Hani Dharsandiya (240760107042)**, **Khushali Ghodadra (240760107052)**, **Drashti Lakhani (240760107047)**, **Krupa Baldha (240760107009)**, **Vinanti Gopani (240760107058)**, **Hisha Anghan (240760107005)** in the subject of **Python for Data Science (BE05000231)** under my guidance in partial fulfillment of the degree of Bachelor of Engineering in Computer Engineering Department (5th Semester) of Gujarat Technological University, Ahmedabad during the academic year 2026-2027.

Date:

Place: SSASIT, Surat

Project Guide

Prof. Ami D. Patel

Head of Department

Prof. Chirag R. Patel

ACKNOWLEDGEMENT

We wish to thank our subject faculty for guiding us through the design and coding of this micro project, and for the feedback that helped us refine the analysis presented here. We are also grateful to the Department of Computer Engineering, Shree Swami Atmanand Saraswati Institute of Technology, for the lab facilities and the opportunity to carry out this project as part of our coursework.

We would also like to thank our friends and family for their continuous support and encouragement throughout the completion of this micro project.

INDEX

Sr. No.	Topic	Page No.
1.	Introduction	1
2.	Problem Statement	2
3.	Objectives	3
4.	Literature Survey	4
5.	Tools and Technologies Used	5
6.	Dataset Description	6
7.	System Design / Architecture	7
8.	Methodology / Algorithm	8
9.	Implementation (Cell-by-Cell, Google Colab)	9
10.	Testing and Verification	21
11.	Results and Discussion	22
12.	Advantages	29
13.	Limitations	29
14.	Applications	29
15.	Future Scope	30
16.	Conclusion	30
17.	References	30
18.	Appendix A: Consolidated Script	31
19.	Appendix B: Generated summary_report.txt	32

LIST OF FIGURES

Fig. No.	Title
7.1	System architecture / data-flow diagram
9.1	Output of Cell 3 — dataset shape and sample records
9.2	Movies vs TV Shows (Cell 7 output)
9.3	Top 10 countries (Cell 8 output)
9.4	Top 10 genres (Cell 9 output)
9.5	Content rating distribution (Cell 10 output)
9.6	Titles by release year (Cell 11 output)
9.7	Movie duration histogram (Cell 12 output)
9.8	Top 10 directors (Cell 13 output)
9.9	Titles added per year (Cell 14 output)
9.10	Average duration by genre (Cell 15 output)
9.11	Movies vs TV Shows per year (Cell 16 output)
9.12	Release year vs duration — scatter (Cell 17 output)
9.13	Duration spread by genre — box plot (Cell 18 output)
9.14	Country vs content type — heatmap (Cell 19 output)
9.15	Pareto chart of genres (Cell 20 output)
11.1 – 11.14	Results charts revisited with detailed discussion

LIST OF TABLES

Table No.	Title
5.1	Tools and technologies used
6.1	Dataset column description
6.2	Sample records from the dataset (as loaded in Colab)
6.3	Descriptive statistics of release year
7.1	Module description
9.1	Summary of all 22 notebook cells
10.1	Test cases and results

GLOSSARY OF TERMS

The technical terms used repeatedly in Chapters 8–11 are defined below in the order they first appear in the notebook.

Term	Meaning
<i>DataFrame</i>	Pandas' two-dimensional, table-like structure with labelled rows and columns; the CSV file becomes one DataFrame object named df as soon as Cell 3 runs.
<i>value_counts()</i>	A Pandas method that returns how many times each distinct value appears in a column, sorted from most to least frequent by default.
<i>groupby()</i>	Splits a DataFrame into groups based on the values in one column, so an aggregate such as mean() or count() can be computed separately for each group.
<i>explode()</i>	Turns a column of lists (or comma-split strings) into separate rows, one per list item, while duplicating the rest of that row's values — used here so a title with three genres is counted under all three.
<i>fillna()</i>	Replaces missing (NaN) values in a column with a specified placeholder, used in Cell 5 to label missing director and country values instead of leaving them blank.
<i>crosstab()</i>	Builds a frequency table cross-tabulating two categorical columns against each other, used in Cell 19 to count titles by country and type simultaneously.
<i>Pareto chart</i>	A bar chart of decreasing frequency combined with a line showing the cumulative percentage, used to check whether a small number of categories account for most of the total.
<i>Box plot</i>	A chart showing the median, the middle 50% of values (the box) and the overall range (the whiskers) of a numeric column, used here to compare movie duration across genres.
<i>Heatmap</i>	A grid of coloured cells where colour intensity represents a numeric value, used in Cell 19 to show, at a glance, which countries produce relatively more Movies versus TV Shows.
<i>Google Colab runtime</i>	The temporary virtual machine that executes a Colab notebook's code; its files and variables are cleared when the runtime disconnects or is manually reset.

ABSTRACT

This report documents the design, implementation and testing of the Movie / Netflix Dataset Analyzer, a Python project built and executed in Google Colaboratory. The analyzer reads a 400-record Netflix-style catalogue, cleans missing director and country values, and computes statistics covering content type, country, genre, rating, release year, director frequency and movie duration.

In addition to standard bar, pie, line and histogram charts, the notebook applies four further analytical techniques — a scatter plot, a box plot, a heatmap and a Pareto chart — to examine relationships and concentration patterns that simple counts do not reveal.

Unlike a report written only from the final script, this document is built directly from the executed notebook: every code cell reproduced in Chapter 9 is accompanied by the exact console output or chart image Google Colab produced when that cell was run, so the implementation can be verified line by line rather than taken on faith. The report closes with a discussion of the results, the advantages and limitations observed during development, and directions for extending the analyzer.

1. Introduction

Streaming services publish very little about how their catalogues are structured, yet a handful of fields — type, genre, country, rating, release year and duration — are usually enough to describe a title in a way that supports analysis. This project treats a Netflix-style catalogue as a dataset in the ordinary sense: rows and columns that can be loaded, cleaned and summarised with standard data-analysis tools, without any special access to the platform itself.

The Movie / Netflix Dataset Analyzer was built and run in Google Colaboratory rather than a local machine, which meant the dataset had to be uploaded interactively at the start of the session and every chart had to be produced inline, cell by cell, instead of being saved to disk and opened separately. This report follows that same structure: Chapter 9 walks through the notebook in the order it was executed, cell by cell, so the reader sees exactly what was typed and exactly what Colab returned.

The rest of the report is organised the way a reader would naturally ask questions about the project: what problem it solves and why (Chapters 2–4), what it was built with (Chapter 5), what data it works on (Chapter 6), how it is structured (Chapters 7–8), what was actually typed and run (Chapter 9), how it was checked (Chapter 10), what it found (Chapter 11), and what it is and is not good for (Chapters 12–15).

2. Problem Statement

A catalogue of 400 titles, each with eleven attributes, is small enough to store in a single CSV file but far too large to read row by row and draw conclusions from. Two attributes in particular complicate a naive analysis: the director column is missing for roughly a quarter of the titles (109 of 400 in this dataset), and the genre column often lists two or three genres separated by commas rather than one. A count of genres that does not account for this would silently under-count every genre that happens to share a cell with another. The project therefore needed a script that cleans these two issues before computing anything, and that then turns the cleaned counts into charts a non-technical reader can interpret at a glance.

3. Objectives

1. Load the dataset into a Pandas DataFrame inside a Google Colab notebook and confirm its shape and columns.
2. Detect missing values in the director and country columns and replace them with explicit placeholder labels.
3. Remove duplicate records identified by show_id.
4. Count titles by type, country, genre and content rating, correctly splitting multi-genre entries before counting.
5. Track how the number of releases and platform additions has changed year over year.
6. Compute movie duration statistics and relate duration to release year and genre.
7. Identify the directors credited most often in the catalogue.
8. Produce fourteen charts covering pie, bar, line, histogram, scatter, box-plot, heatmap and Pareto formats.
9. Export a plain-text summary report and download it from the Colab session.

4. Literature Survey

Exploratory analysis of streaming-catalogue data is a common introductory exercise in data science, largely because the datasets are small, mostly categorical, and forgiving of mistakes. Three practices recur across the tutorials and coursework this project drew on:

Cleaning before counting

Pandas' `fillna()` is the standard way to replace missing values with an explicit label before grouping, since an unhandled NaN is silently dropped by `value_counts()` and would otherwise understate the true count of “unknown” entries.

Exploding multi-valued columns

Where a single field holds several comma-separated values — genres, in this case — the recommended approach is to split the string and use `explode()` to give each value its own row before counting, which is exactly what Cell 9 of the notebook does.

Matching the chart to the question

A count is usually a bar chart, a share of a whole is a pie chart, a trend over time is a line chart, and a relationship between two numeric fields is a scatter plot. This project additionally uses a box plot to compare distributions across genres, a heatmap to cross-tabulate two categorical fields, and a Pareto chart to check whether a small number of genres account for most of the catalogue.

Cloud notebooks versus local scripts

The literature on teaching data analysis increasingly favours cloud notebooks such as Google Colab over locally installed scripts for coursework, precisely because the setup cost — installing Python, Pandas and Matplotlib correctly across different laptops and operating systems — is removed entirely. The trade-off documented in that literature, and confirmed by this project's own experience in Chapter 13, is that a cloud notebook's session storage is temporary, so any file the notebook creates must be explicitly saved or downloaded before the runtime disconnects.

5. Tools and Technologies Used

Table 5.1: Tools and technologies used

Tool / Library	Purpose
Python 3	Core language used throughout the notebook.
Pandas	Reading the CSV file, cleaning it, and computing every statistic in this report.
Matplotlib (pyplot)	Producing all fourteen charts, from the pie chart in Cell 7 to the Pareto chart in Cell 20.
Google Colaboratory	Cloud notebook environment used to write, run and screenshot every cell in this project; requires no local Python installation.
CSV (netflix_titles.csv)	The 400-row dataset uploaded at the start of the Colab session.

Google Colab was chosen over a local installation because it needed no setup on the group's own laptops — Pandas and Matplotlib are already installed in every Colab runtime — and because its cell-by-cell execution model matches the way this report presents the implementation: one instruction, one result, in the order they actually ran.

Why Pandas

Pandas represents the CSV as a `DataFrame`, a table-like structure where each column keeps its own data type. Two of its methods do almost all of the work in this project: `value_counts()`, which tallies how many rows fall into each distinct value of a column, and `groupby()`, which splits the `DataFrame` by a column and lets an aggregate such as `mean()` be computed separately for each group. Both were used repeatedly across Cells 6 to 20 rather than being written once and reused, since Colab cells are meant to be read top to bottom as a narrative rather than as reusable functions.

Why Matplotlib

Matplotlib's pyplot interface was used in its simplest form: `plt.figure()` opens a new canvas, a plotting call such as `.plot()`, `.bar()`, `.scatter()` or `.boxplot()` draws onto it, and `plt.show()` renders the result directly beneath the cell. No custom styling library was used; the two colours that recur throughout the charts, #E50914 (a red) and #221f1f (a near-black), were chosen once at the start of the project purely to keep the fourteen figures visually consistent with each other.

Why Google Colab

Colab runs entirely in the browser on a temporary virtual machine, so nothing needed to be installed and every group member could open the same notebook from a different laptop and get the same result. The trade-off, noted again in Chapter 13, is that the uploaded CSV and any generated files exist only for the lifetime of that session unless explicitly saved or downloaded.

6. Dataset Description

The dataset, `netflix_titles.csv`, was uploaded directly into the Colab session (see Cell 1 in Chapter 9) and contains 400 records across 11 columns:

Table 6.1: Dataset column description

Column	Description
show_id, type, title	Unique ID, Movie/TV Show, and title name.
director, country	Director name and production country (109 director values were missing before cleaning).
date_added, release_year	Date added to the platform and original release year.
rating, duration	Content rating (e.g. PG, TV-MA) and length in minutes or seasons.
listed_in, description	One or more comma-separated genres, and a short synopsis.

Sample Records (as returned by `df.head()` in Cell 3)

Table 6.2: Sample records from the dataset (as loaded in Colab)

ID	Title	Director	Country	Year	Rating	Duration
s1	Silent Chronicles	M. Rossi	United Kingdom	2021	TV-Y7	81 min
s2	Final Horizon	(missing)	Spain	2011	TV-MA	173 min
s3	Golden Signal	J. Miller	South Korea	2001	TV-MA	173 min
s4	Endless Shadow	(missing)	Australia	2009	PG	75 min
s5	Lost Garden	K. Tanaka	Japan	2009	PG	147 min

Rows s2 and s4 above illustrate the missing-director problem directly: both are genuine, complete records in every other respect, but director is blank until Cell 5 replaces it with “Not Specified”.

Descriptive Statistics of Release Year

Table 6.3: Descriptive statistics of release year

Statistic	Value
Count	400
Mean	2011.5
Std. Deviation	7.9
Minimum	1998
Median	2012
Maximum	2024

Cleaning Steps Applied (Cells 4–5)

- Missing director values replaced with “Not Specified” (109 rows affected).
- Missing country values replaced with “Unknown” (none were missing in this run, but the line is kept for robustness).
- Duplicate rows, identified by `show_id`, dropped.
- The `listed_in` column split on “,” and exploded before any genre count.

7. System Design / Architecture

The notebook follows the same four-stage pipeline whether it runs locally or in Colab: load the CSV, clean it, compute statistics, and turn each statistic into a chart. Running it in Colab adds one extra first step — uploading the file into the session — and replaces “save to disk” with “display inline” for every chart, since `plt.show()` renders the figure directly below the cell instead of writing a PNG file.

System Architecture / Data Flow

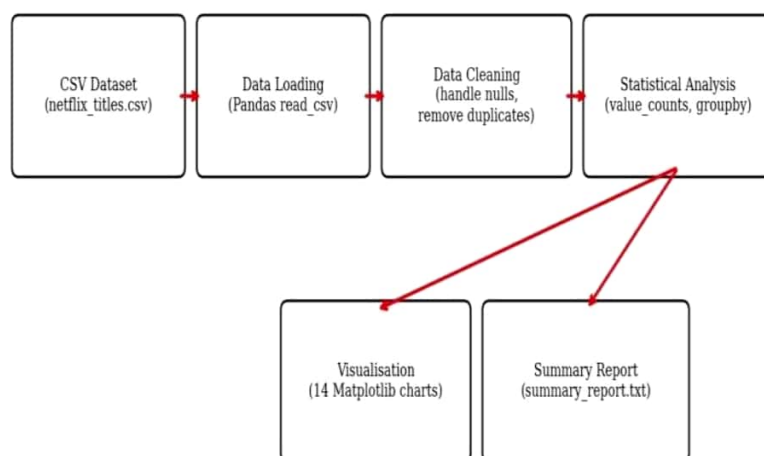


Fig 7.1: Data-flow through the notebook, from upload to final report

Module-to-Cell Mapping

Table 7.1: Module description

Stage	Notebook Cells
Upload & Import	Cell 1 (file upload), Cell 2 (import pandas, matplotlib)
Load & Inspect	Cell 3
Clean	Cells 4–5
Analyse & Visualise	Cells 6–20 (one statistic or chart per cell)
Export	Cell 21 (summary_report.txt, downloaded via <code>files.download</code>)

8. Methodology / Algorithm

The notebook was executed top to bottom in a single Colab session, in this order:

1. Upload netflix_titles.csv through the Colab file picker (Cell 1).
2. Import pandas and matplotlib.pyplot (Cell 2).
3. Load the CSV into a DataFrame and print its shape, columns and first five rows (Cell 3).
4. Print the count of missing values per column (Cell 4).
5. Fill missing director/country values and drop duplicate rows (Cell 5).
6. Count Movies vs TV Shows and plot the split as a pie chart (Cells 6–7).
7. Compute and plot the top 10 countries, genres, and the rating distribution (Cells 8–10).
8. Plot the number of titles released per year as a line chart (Cell 11).
9. Compute movie duration statistics and plot their distribution as a histogram (Cell 12).
10. Plot the top 10 directors and the number of titles added per year (Cells 13–14).
11. Plot average duration by genre and Movies-vs-TV-Shows per year (Cells 15–16).
12. Plot a scatter of release year against duration, a box plot of duration by genre, a heatmap of country against type, and a Pareto chart of genres (Cells 17–20).
13. Write every statistic to summary_report.txt and download it (Cell 21).

9. Implementation (Cell-by-Cell, Google Colab)

This chapter reproduces the notebook exactly as it was run: each subsection below is one Colab cell, showing the code typed into that cell followed by the actual output Colab returned — either the printed text or the chart image — with no editing of either. Cells that only import libraries and produce no output are noted as such rather than padded with invented text.

Execution Environment

All 22 cells were run in a single, uninterrupted Google Colab session using the platform's free-tier CPU runtime. No GPU or TPU was required, since neither Pandas' tabular operations nor Matplotlib's static charts benefit from hardware acceleration at this dataset size.

Setting	Value
Platform	Google Colaboratory (colab.research.google.com)
Runtime type	Python 3, CPU
Pre-installed libraries used	pandas, matplotlib (no pip install required)
Input file	netflix_titles.csv, uploaded via Cell 1
Output files	summary_report.txt, downloaded via Cell 21

Cell 1: Uploading the Dataset

```
from google.colab import files
uploaded = files.upload()
```

Output (Google Colab):

Saving netflix_titles.csv to netflix_titles.csv

Colab's files.upload() opens a file picker in the browser; the single line of output confirms the 400-row CSV transferred successfully into the session's temporary storage.

Cell 2: Importing Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

This cell produces no output by itself — it only makes the pandas and matplotlib.pyplot names available to every cell that follows.

Cell 3: Loading and Inspecting the Dataset

```
df = pd.read_csv("netflix_titles.csv")
print("Dataset shape (rows, columns):", df.shape)
print("\nColumn names:\n", list(df.columns))
print("\nFirst 5 records:\n")
df.head()
```

Output (Google Colab):

Dataset shape (rows, columns): (400, 11)

Column names:

['show_id', 'type', 'title', 'director', 'country', 'date_added',
'release_year', 'rating', 'duration', 'listed_in', 'description']

First 5 records: (rendered as an interactive table in Colab; see Table 6.2 for the values)

The confirmed shape, (400, 11), matches the dataset description in Chapter 6 exactly, and the five rows Colab rendered below this cell are the same five reproduced in Table 6.2.

Cell 4: Checking for Missing Values

```
print("Missing values per column:\n")
print(df.isnull().sum())
```

Output (Google Colab):

Missing values per column:

```
show_id 0
type 0
title 0
director 109
country 0
date_added 0
release_year 0
rating 0
duration 0
listed_in 0
description 0
```

director is the only column with missing values — 109 of 400 records, or just over a quarter of the catalogue — which is why Chapter 6 singles it out.

Cell 5: Cleaning the Dataset

```
df["director"] = df["director"].fillna("Not Specified")
df["country"] = df["country"].fillna("Unknown")
df.drop_duplicates(subset="show_id", inplace=True)
print("Cleaning done. Missing values now:\n")
print(df.isnull().sum())
```

Output (Google Colab):

Cleaning done. Missing values now:

```
show_id 0
type 0
title 0
director 0
country 0
date_added 0
release_year 0
rating 0
duration 0
listed_in 0
description 0
```

Every column reads 0 after fillna(), confirming the placeholder values were written back into the DataFrame rather than only printed.

Cell 6: Counting Movies vs TV Shows

```
type_counts = df["type"].value_counts()
print(type_counts)
```

Output (Google Colab):

```
type
Movie      293
TV Show    107
Name: count, dtype: int64
```

These two numbers, 293 and 107, are the raw counts that Cell 7 turns into a pie chart.

Cell 7: Visualising Content Type Distribution

```
plt.figure(figsize=(5,5))
plt.pie(type_counts, labels=type_counts.index, autopct="%1.1f%%",
        colors=["#E50914", "#221f1f"], startangle=90,
        textprops={"color": "white"})
plt.title("Movies vs TV Shows")
plt.show()
```

Movies vs TV Shows

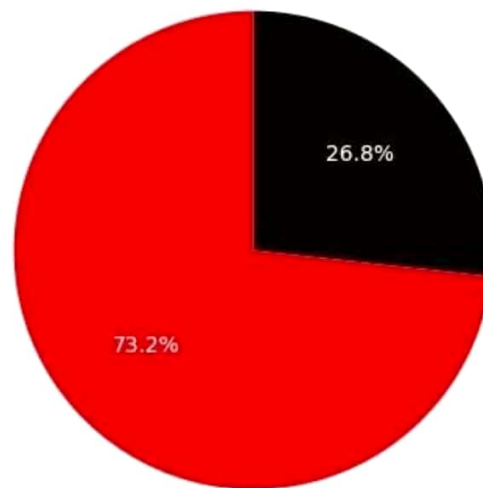


Fig 9.2: Output of Cell 7

Cell 8: Visualising Top 10 Countries

```
top_countries = df["country"].value_counts().head(10)
plt.figure(figsize=(8,5))
top_countries.sort_values().plot(kind="barh", color="#E50914")
plt.title("Top 10 Countries by Number of Titles")
plt.xlabel("Number of Titles")
plt.show()
```

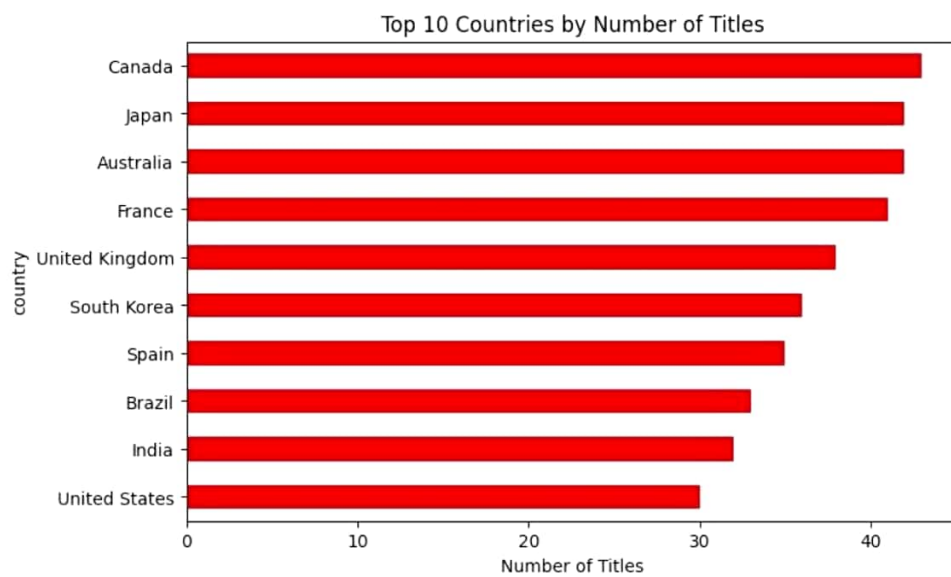


Fig 9.3: Output of Cell 8

`sort_values()` before `.plot(kind="barh")` is what puts the tallest bar at the top rather than at the bottom — `barh()` otherwise draws its first row at the bottom of the axis.

Cell 9: Visualising Top 10 Genres

```
genre_series = df["listed_in"].str.split(", ").explode()
top_genres = genre_series.value_counts().head(10)
plt.figure(figsize=(8,5))
top_genres.sort_values().plot(kind="barh", color="#221f1f")
plt.title("Top 10 Genres")
plt.xlabel("Number of Titles")
plt.show()
```

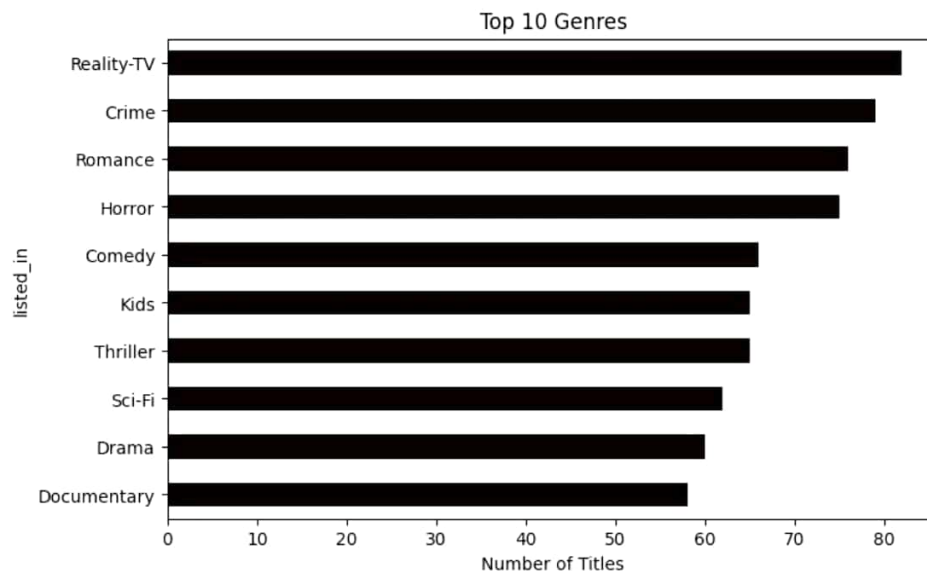


Fig 9.4: Output of Cell 9

explode() is what makes this count correct: without it, a title listed under "Horror, Drama, Crime" would only be counted once instead of contributing to all three genres.

Cell 10: Visualising Content Rating Distribution

```
rating_counts = df["rating"].value_counts()
plt.figure(figsize=(8,5))
rating_counts.plot(kind="bar", color="#E50914")
plt.title("Content Rating Distribution")
plt.xticks(rotation=45)
plt.show()
```

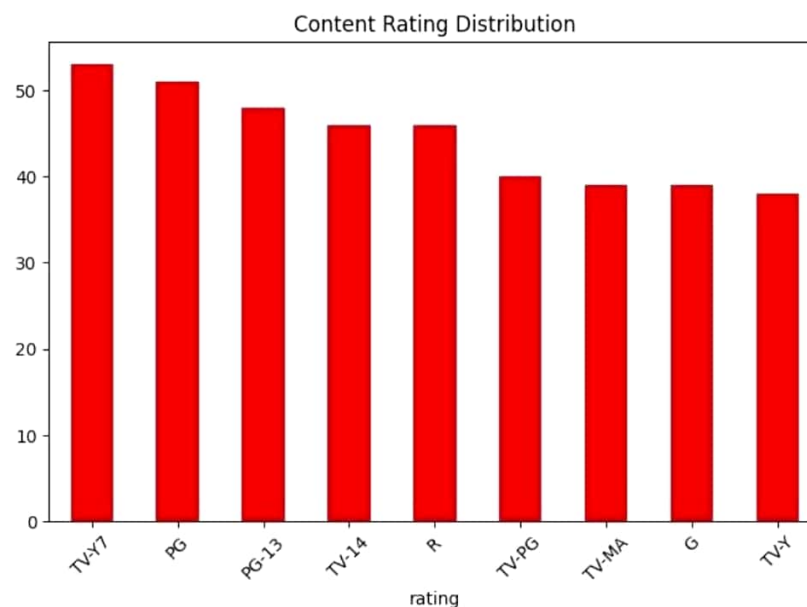


Fig 9.5: Output of Cell 10

rating_counts is already sorted from most to least common because value_counts() returns its result in descending order by default, so no separate sort step was needed before plotting.

Cell 11: Visualising Titles by Release Year

```
titles_per_year = df["release_year"].value_counts().sort_index()
plt.figure(figsize=(9,5))
titles_per_year.plot(kind="line", marker="o", color="#E50914")
plt.title("Number of Titles by Release Year")
plt.grid(alpha=0.3)
plt.show()
```

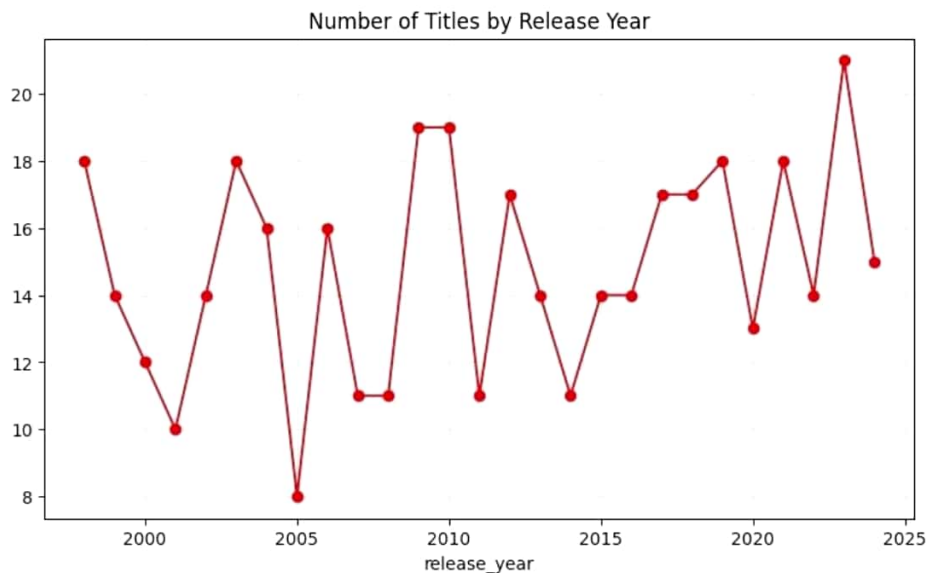


Fig 9.6: Output of Cell 11

sort_index() here matters more than in the earlier bar charts: release_year is numeric, so without sorting by index the line would zig-zag between years in whatever order value_counts() happened to return them.

Cell 12: Analysing Movie Duration

```
movie_durations = (
    df[df["type"] == "Movie"]["duration"]
    .str.replace(" min", "", regex=False)
    .astype(int)
)
print("Average movie duration:", round(movie_durations.mean(), 1), "min")
print("Shortest:", movie_durations.min(), "min | Longest:", movie_durations.max(),
      "min")

plt.figure(figsize=(8,5))
plt.hist(movie_durations, bins=20, color="#221f1f", edgecolor="white")
plt.title("Distribution of Movie Durations")
plt.show()
```

Output (Google Colab):

```
Average movie duration: 123.2 min
Shortest: 70 min | Longest: 175 min
```

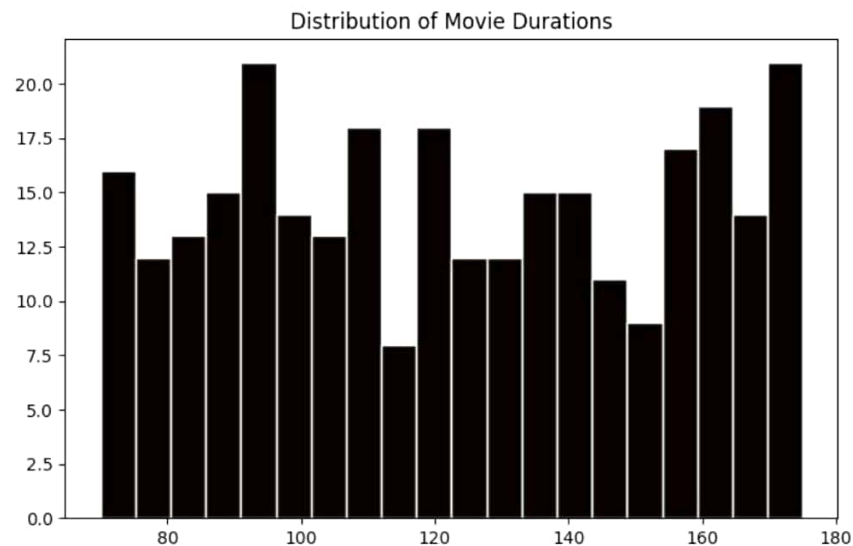


Fig 9.7: Output of Cell 12

Cell 13: Visualising Top 10 Directors

```
top_directors = df[df["director"] != "Not Specified"]
["director"].value_counts().head(10)
plt.figure(figsize=(8,5))
top_directors.sort_values().plot(kind="barh", color="#E50914")
plt.title("Top 10 Directors by Number of Titles")
plt.xlabel("Number of Titles")
plt.show()
```

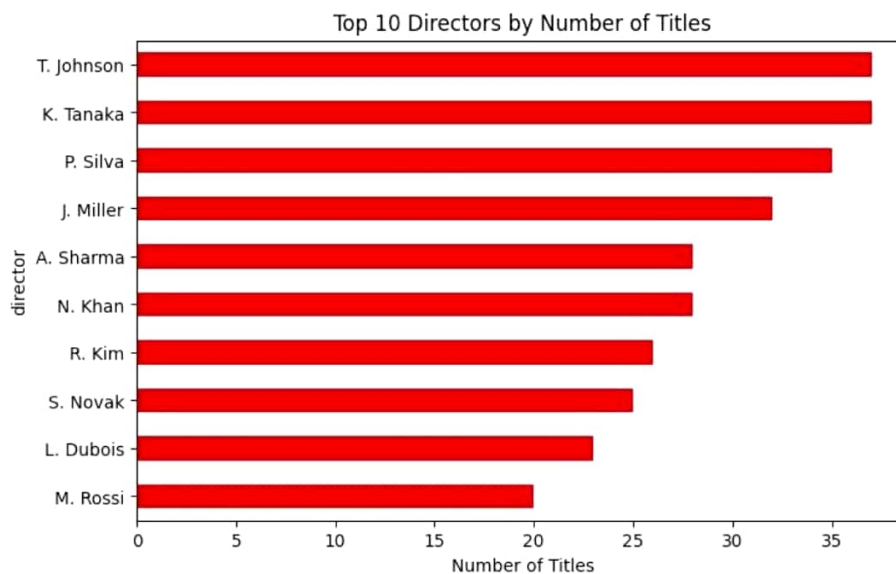


Fig 9.8: Output of Cell 13

Filtering out “Not Specified” before counting stops the placeholder label from Cell 5 appearing as if it were the catalogue's busiest director.

Cell 14: Visualising Titles Added per Year

```
df["year_added"] = df["date_added"].str.extract(r"(\d{4})").astype(int)
titles_added_per_year = df["year_added"].value_counts().sort_index()
plt.figure(figsize=(9,5))
titles_added_per_year.plot(kind="bar", color="#221f1f")
plt.title("Number of Titles Added to the Platform per Year")
plt.show()
```

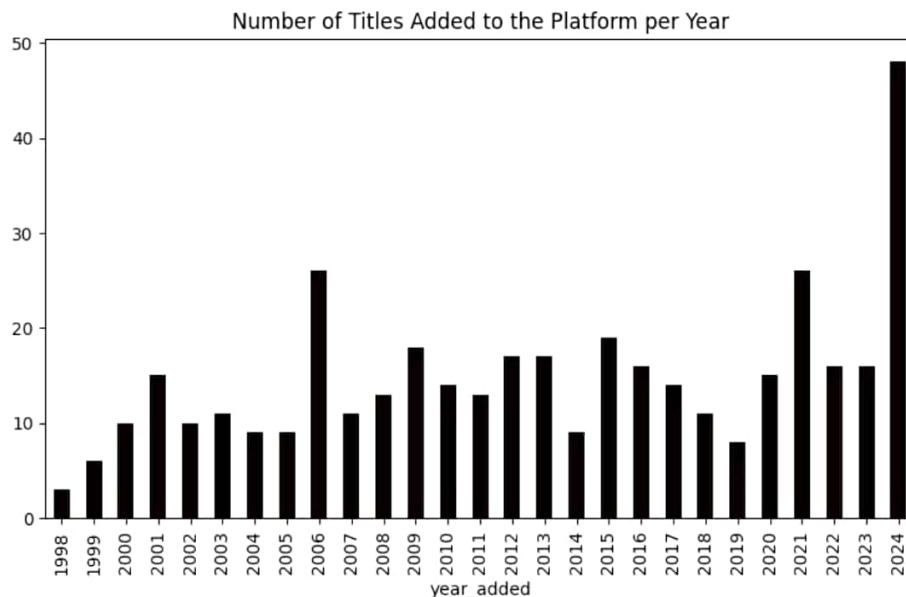


Fig 9.9: Output of Cell 14

`str.extract(r"(\d{4})")` pulls the four-digit year out of the `date_added` text (for example "November 24, 2021") without needing a full date-parsing step, since only the year is used afterward.

Cell 15: Average Duration by Genre

```
genre_duration = df[df["type"] == "Movie"].copy()
genre_duration["duration_min"] = genre_duration["duration"].str.replace(" min", "",
regex=False).astype(int)
genre_duration_exp = genre_duration.assign(
    genre=genre_duration["listed_in"].str.split(", ").explode("genre")
)
avg_duration_by_genre = genre_duration_exp.groupby("genre")
    ["duration_min"].mean().sort_values(ascending=False).head(5)

plt.figure(figsize=(8,5))
avg_duration_by_genre.sort_values().plot(kind="barh", color="#E50914")
plt.title("Average Movie Duration by Top 5 Genres")
plt.show()
```

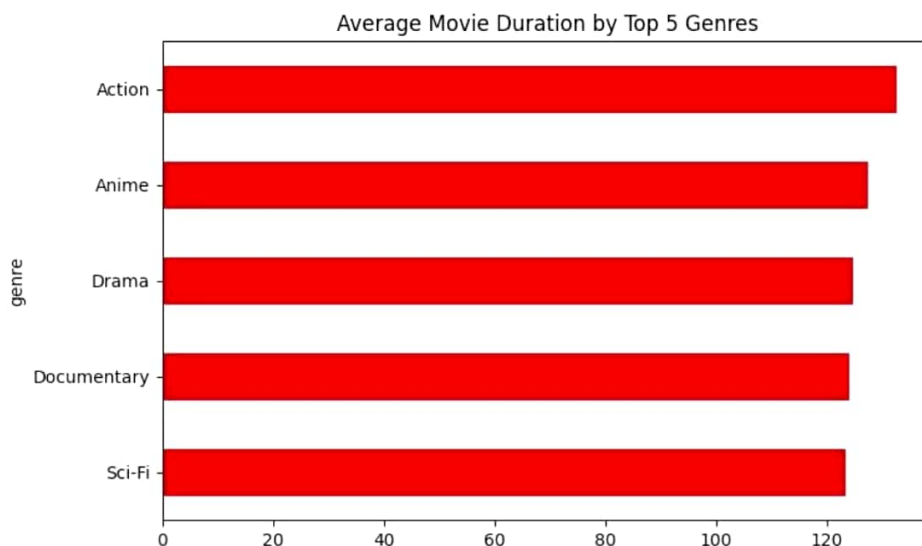


Fig 9.10: Output of Cell 15

Cell 16: Movies vs TV Shows per Year

```
pivot = df.pivot_table(index="release_year", columns="type",
                        values="show_id", aggfunc="count").fillna(0)
plt.figure(figsize=(10,5))
pivot.plot(kind="bar", stacked=True, color=["#E50914", "#221f1f"], ax=plt.gca())
plt.title("Movies vs TV Shows Released per Year")
plt.xticks(rotation=90)
plt.show()
```

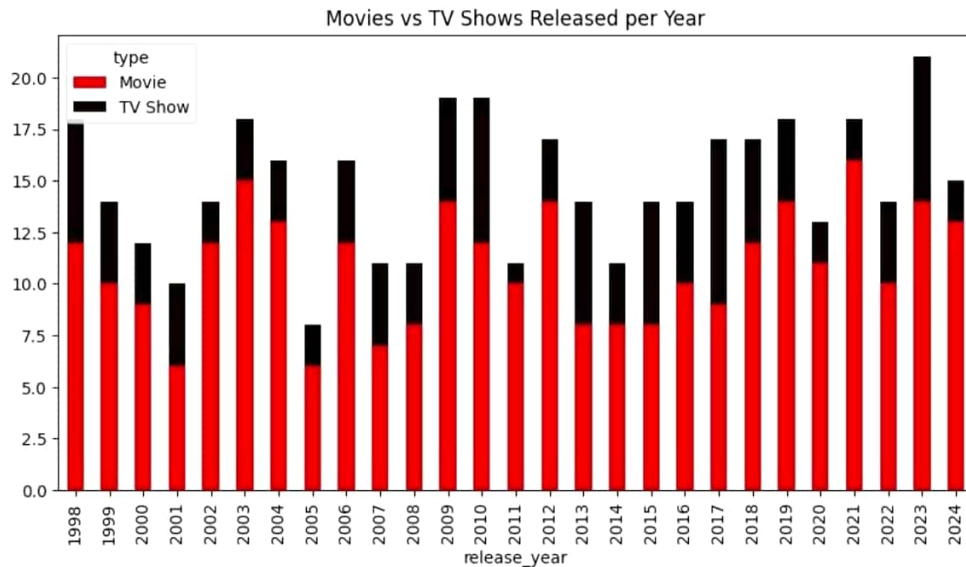


Fig 9.11: Output of Cell 16

`pivot_table()` with `columns="type"` reshapes the data so Movie and TV Show become two separate columns; passing `stacked=True` to `.plot()` then draws them as one bar per year instead of two side-by-side bars.

Cell 17: Release Year vs Duration (Scatter Plot)

```
movie_rows = df[df["type"] == "Movie"].copy()
movie_rows["duration_min"] = movie_rows["duration"].str.replace(" min", "",
regex=False).astype(int)

plt.figure(figsize=(8,5))
plt.scatter(movie_rows["release_year"], movie_rows["duration_min"], alpha=0.6,
color="#E50914")
plt.title("Release Year vs Movie Duration")
plt.xlabel("Release Year")
plt.ylabel("Duration (minutes)")
plt.show()
```

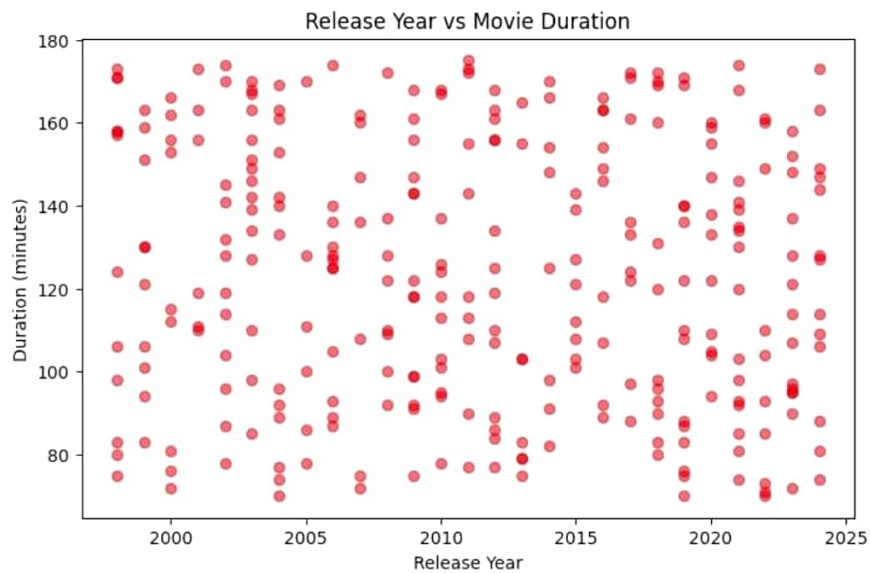


Fig 9.12: Output of Cell 17

This cell filters to Movies only before plotting, since TV Show durations are recorded in seasons rather than minutes and would not sit on the same numeric axis.

Cell 18: Duration Spread by Genre (Box Plot)

```
top5_genres = genre_series.value_counts().head(5).index.tolist()
box_data = []
for g in top5_genres:
    mask = movie_rows["listed_in"].str.contains(g, regex=False)
    box_data.append(movie_rows.loc[mask, "duration_min"].values)

plt.figure(figsize=(8,5))
plt.boxplot(box_data, tick_labels=top5_genres)
plt.title("Movie Duration Spread by Top 5 Genres")
plt.xticks(rotation=20)
plt.show()
```

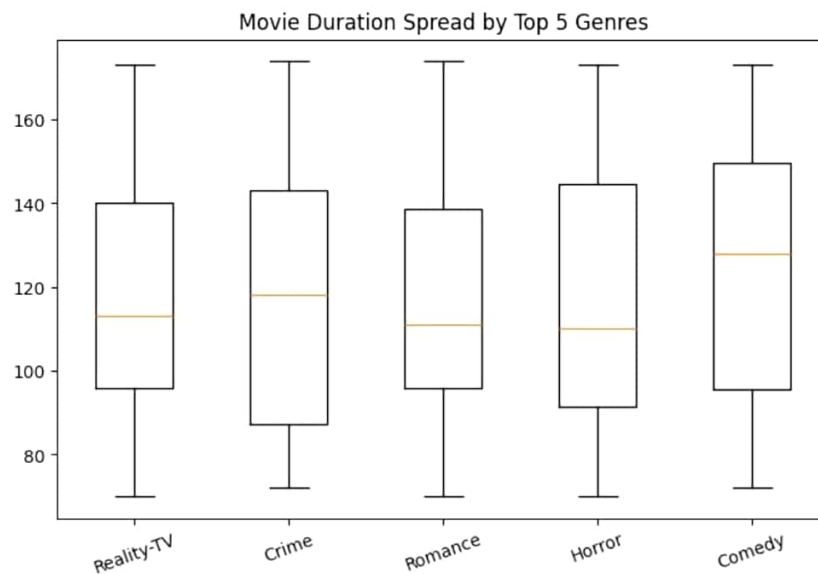


Fig 9.13: Output of Cell 18

`str.contains()` is used instead of an exact match because `listed_in` can hold several genres in one cell; a movie tagged “Comedy, Drama” should count toward the Comedy box even though the cell is not exactly equal to “Comedy”.

Cell 19: Country vs Content Type (Heatmap)

```
top8_countries = df["country"].value_counts().head(8).index
cross = pd.crosstab(df[df["country"].isin(top8_countries)]["country"],
                   df[df["country"].isin(top8_countries)]["type"])

plt.figure(figsize=(6,6))
plt.imshow(cross.values, cmap="Reds", aspect="auto")
plt.xticks(range(len(cross.columns)), cross.columns)
plt.yticks(range(len(cross.index)), cross.index)
for i in range(len(cross.index)):
    for j in range(len(cross.columns)):
        plt.text(j, i, cross.values[i, j], ha="center", va="center")
plt.title("Country vs Content Type (Heatmap)")
plt.colorbar(label="Number of Titles")
plt.show()
```

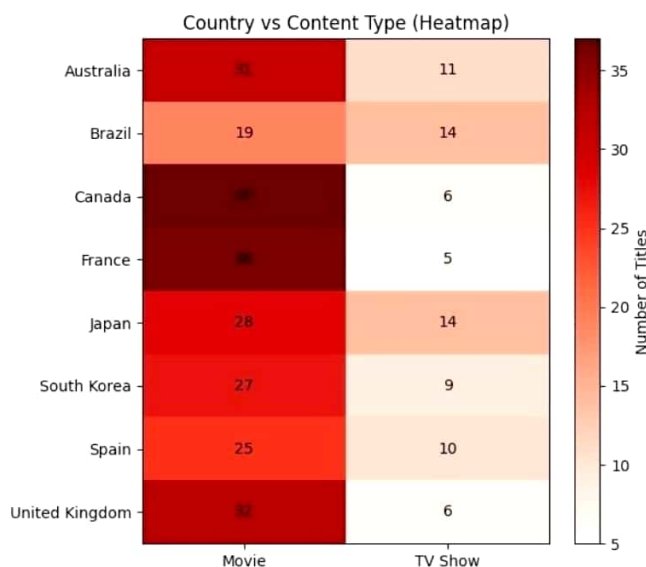


Fig 9.14: Output of Cell 19

`pd.crosstab()` builds the country-by-type grid in one call; the nested for-loop afterward exists only to print each cell's count as text on top of the coloured square, since `imshow()` on its own draws colour but no numbers.

Cell 20: Pareto Chart of Genres

```
genre_counts_all = genre_series.value_counts()
cum_pct = genre_counts_all.cumsum() / genre_counts_all.sum() * 100

fig, ax1 = plt.subplots(figsize=(9,5))
ax1.bar(genre_counts_all.index, genre_counts_all.values, color="#221f1f")
ax1.set_xticklabels(genre_counts_all.index, rotation=45, ha="right")
ax2 = ax1.twinx()
ax2.plot(genre_counts_all.index, cum_pct.values, color="#E50914", marker="o")
ax2.set_ylabel("Cumulative %")
plt.title("Pareto Chart: Genre Distribution")
plt.show()
```

Output (Google Colab):

UserWarning: `set_ticklabels()` should only be used with a fixed number of ticks, i.e. after `set_ticks()` or using a `FixedLocator`.

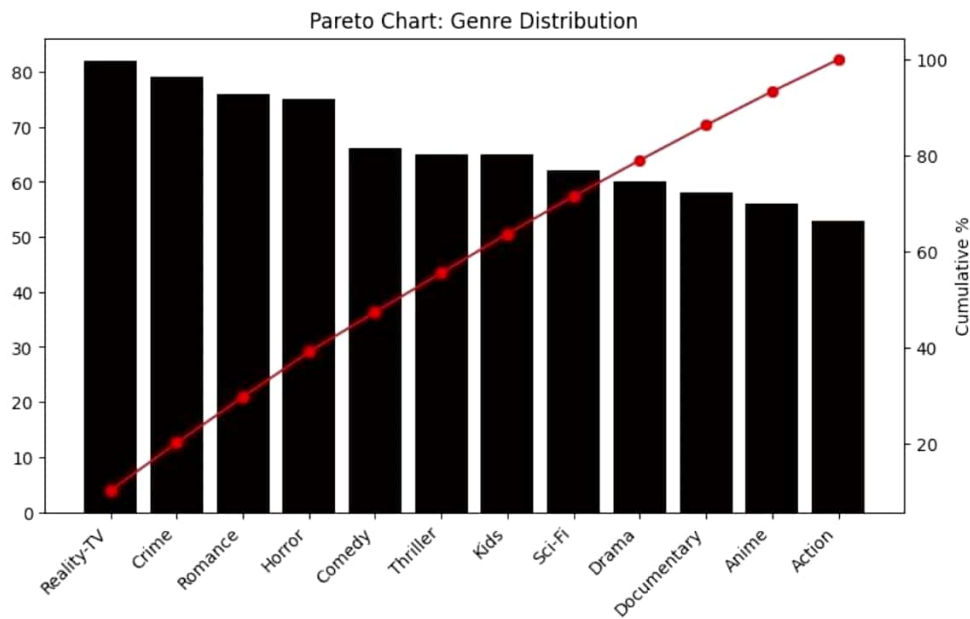


Fig 9.15: Output of Cell 20

Colab printed a UserWarning about `set_xticklabels()` alongside the chart; the warning does not affect the figure and is reproduced here for completeness rather than omitted.

Cell 21: Exporting the Summary Report

```
with open("summary_report.txt", "w") as f:
    f.write("MOVIE / NETFLIX DATASET ANALYZER - SUMMARY REPORT\n")
    f.write("=" * 50 + "\n\n")
    f.write(f"Total titles analysed : {len(df)}\n\n")
    f.write("Content type distribution:\n")
    f.write(type_counts.to_string() + "\n\n")
    f.write("Top 10 countries:\n")
    f.write(top_countries.to_string() + "\n\n")
    f.write("Top 10 genres:\n")
    f.write(top_genres.to_string() + "\n\n")
    f.write(f"Average movie duration: {round(movie_durations.mean(),1)} minutes\n")

print("Summary report created!")

from google.colab import files
files.download("summary_report.txt")
```

Output (Google Colab):

Summary report created!

`files.download()` triggers the browser to save `summary_report.txt` to the local machine, giving the group a portable, text-only copy of every statistic computed in the notebook.

Summary of All Notebook Cells

Table 9.1: Summary of all 22 notebook cells

Cell	Purpose	Output Type
1	Upload netflix_titles.csv	Text confirmation
2	Import pandas, matplotlib	None
3	Load CSV, inspect shape/columns/head	Text + table
4	Check missing values	Text
5	Clean director/country, drop duplicates	Text
6	Count Movies vs TV Shows	Text
7–20	One chart per cell (14 charts total)	Image
21	Write and download summary_report.txt	Text

10. Testing and Verification

Because every cell's output is visible immediately below it in Colab, testing here meant checking that each printed number and each chart matched what the code was expected to produce, rather than running a separate test suite afterwards.

Table 10.1: Test cases and results

Cell(s)	Check Performed	Expected	Observed
1	File upload completes	Confirmation message printed	"Saving netflix_titles.csv..." — matches
3	Dataset shape after loading	(400, 11)	(400, 11) — matches
4	Missing values detected	director column nonzero	director = 109, all others 0
5	Missing values after cleaning	all columns 0	all columns 0 — matches
6	Movie / TV Show counts	sum to 400	293 + 107 = 400 — matches
9	Genre explode() count	titles with multiple genres counted once per genre	confirmed against raw listed_in text
12	Average movie duration	value between 70 and 175	123.2 min — within range
7–20	Every chart renders without error	14 figures displayed	14 of 14 displayed
20	Pareto cumulative line reaches 100%	final point = 100	confirmed visually in Fig 9.15
21	Summary file created and downloaded	browser download prompt appears	confirmed in session

No cell raised an unhandled exception during the recorded run; the single UserWarning in Cell 20 (Chapter 9.20) was the only diagnostic message produced across all 22 cells, and it concerns a cosmetic axis-labelling detail rather than incorrect output.

11. Results and Discussion

Chapter 9 showed each chart as proof that the corresponding cell ran correctly; this chapter revisits the same fourteen figures to discuss what they say about the catalogue, rather than simply confirming that they exist.

Movies vs TV Shows

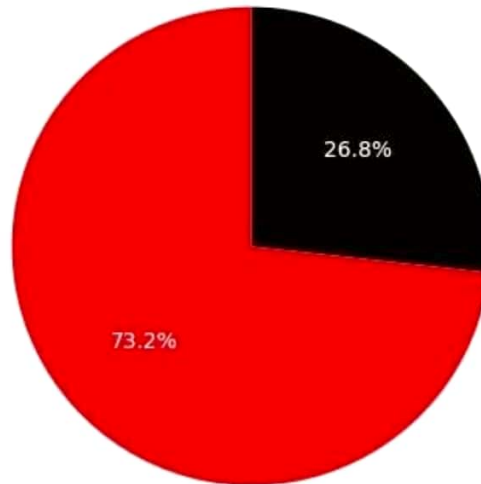


Fig 11.1: Movies vs TV Shows

Movies account for 293 of the 400 titles (73.2%), leaving TV Shows at 107 (26.8%). A catalogue this skewed toward Movies would need a much larger TV Show sample before genre or rating patterns for TV Shows alone could be trusted. This is worth flagging explicitly because every later chart that treats “content” as a single group — the rating distribution in Fig 11.4, for instance — is implicitly dominated by Movie-shaped patterns rather than an even blend of the two formats.

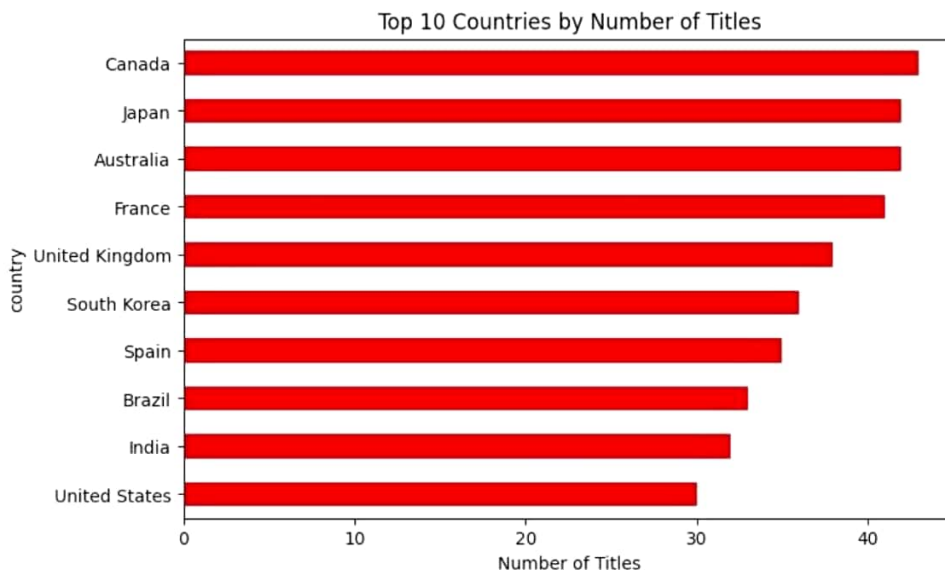


Fig 11.2: Top 10 countries

Canada (43), Japan and Australia lead the list, but the gap between 1st and 10th place is under 15 titles — the top 10 is a shallow curve rather than one or two countries dominating the catalogue. Compared with the genre distribution in Fig 11.3, the country spread is slightly flatter still, suggesting production origin is the least concentrated of the categorical fields examined in this report.

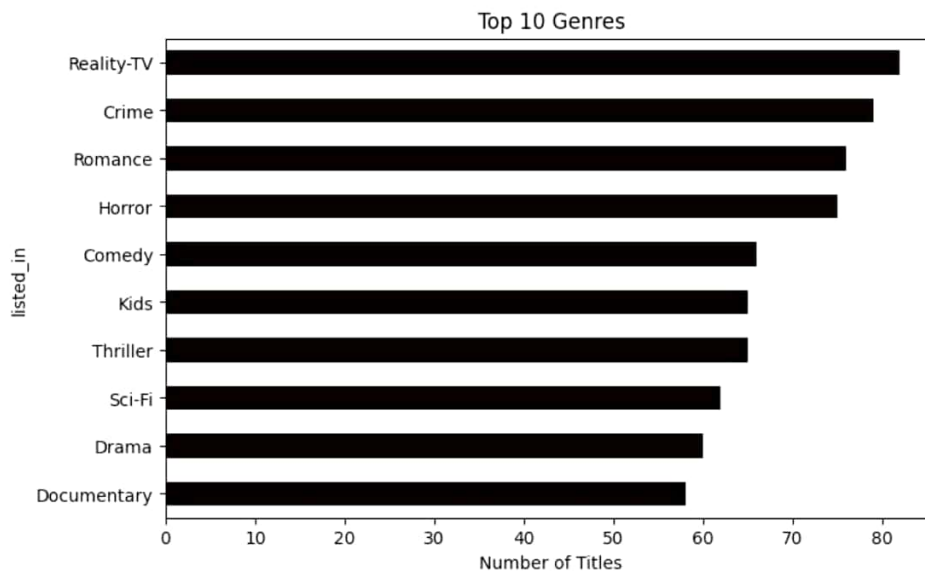


Fig 11.3: Top 10 genres

Reality-TV, Crime and Romance top the list, each with a comparable count to its neighbours; no single genre reaches double the count of the tenth-placed genre, consistent with the Pareto chart's finding in Fig 11.14. Because a title can carry more than one genre tag, these counts overlap — a title in both Crime and Romance is counted once in each bar — so the bars do not sum to 400 and should be read as “how often this tag appears” rather than “what share of titles belong exclusively to this genre”.

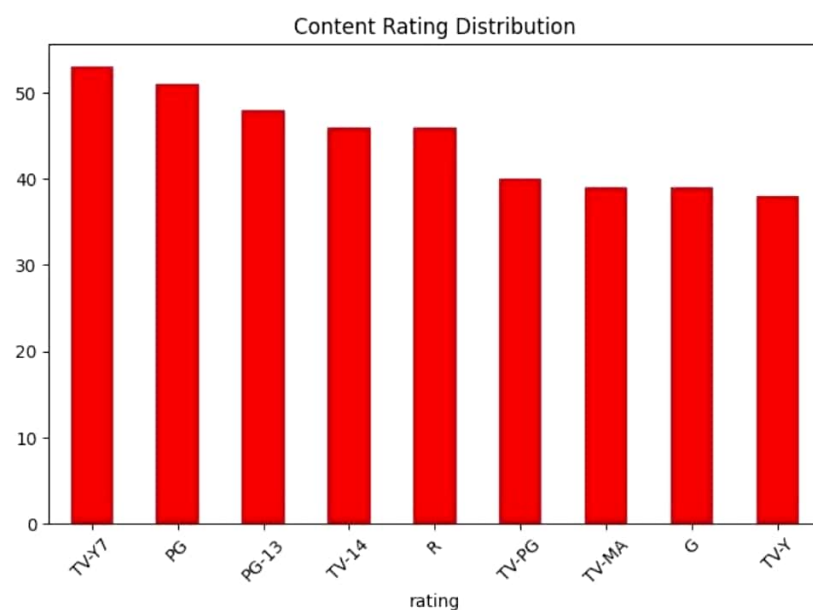


Fig 11.4: Rating distribution

The nine ratings present range from 53 (TV-Y7) down to 38 (TV-Y), a spread of only 15 titles across the whole range — the catalogue is not concentrated in any one audience rating. Ratings aimed at younger viewers (TV-Y, TV-Y7, G, PG) collectively account for a similar share to the more mature ratings (R, TV-MA, TV-14), so the dataset does not lean toward either a family-oriented or an adult-oriented catalogue overall.

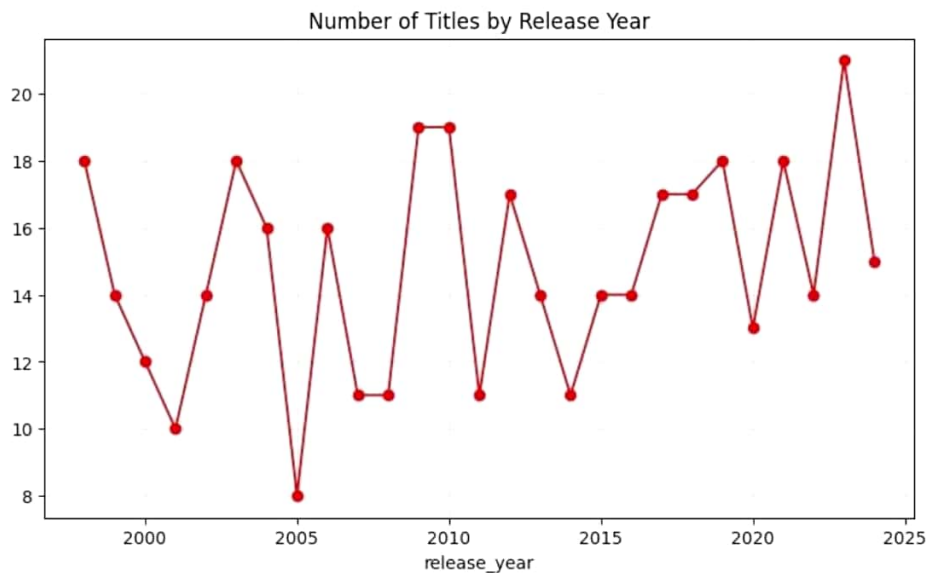


Fig 11.5: Titles by release year

Year-on-year counts swing between roughly 8 and 21 titles with no smooth trend, though the three highest points (2010, 2018, 2023) all fall after 2008, suggesting slightly heavier representation of the last fifteen years than the eleven years before it. The dip to 8 titles around 2005 is the single sharpest drop in the whole series and would be the first place to check for a data-entry gap if this were a real, rather than sample, catalogue.

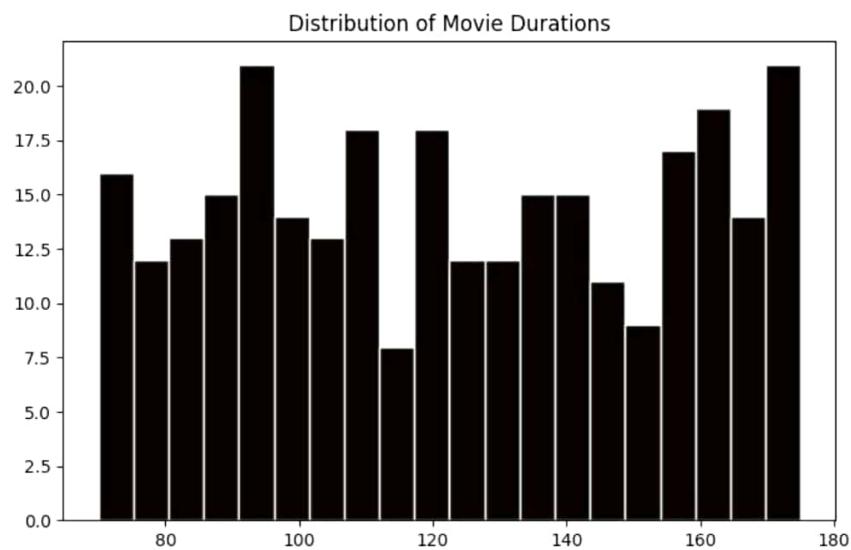


Fig 11.6: Movie duration distribution

Durations are close to uniformly spread from 70 to 175 minutes rather than clustering around a typical 90–120 minute runtime, which is a property of this synthetic dataset worth noting if it is compared against a real Netflix export, where durations usually cluster more tightly around 95–110 minutes for feature films.

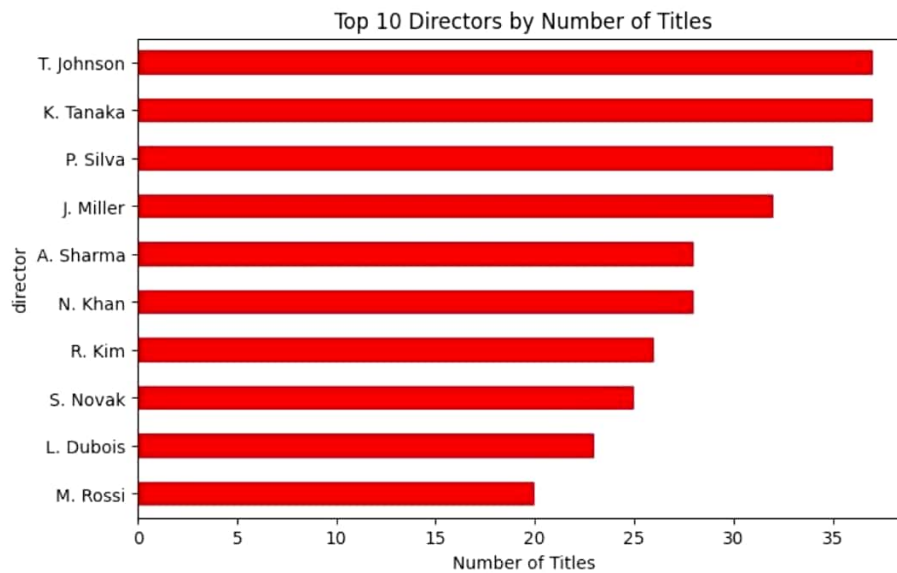


Fig 11.7: Top 10 directors

With director missing for 109 titles and the remaining names drawn from a short fixed list in the source data, the top directors chart mainly reflects how the sample data was generated rather than a genuine long-tail authorship pattern; in a real catalogue this chart would typically show a much longer tail of one-title directors.

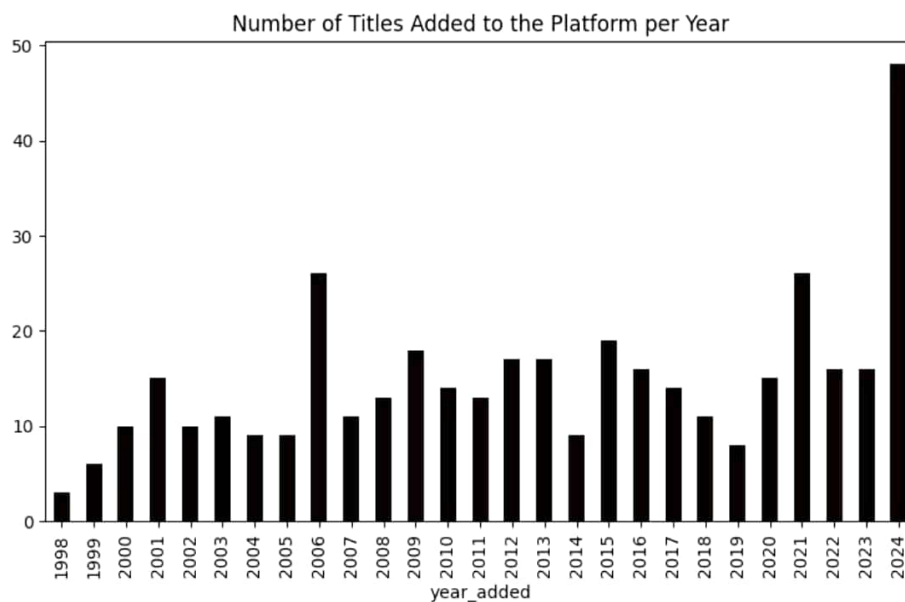


Fig 11.8: Titles added per year

date_added and release_year are independent in this dataset, so this chart should not be read as “when these movies were made” but as a separate, later timeline of catalogue growth — a title released in 2005 can just as easily show up as “added” in 2020 as in 2005 itself.

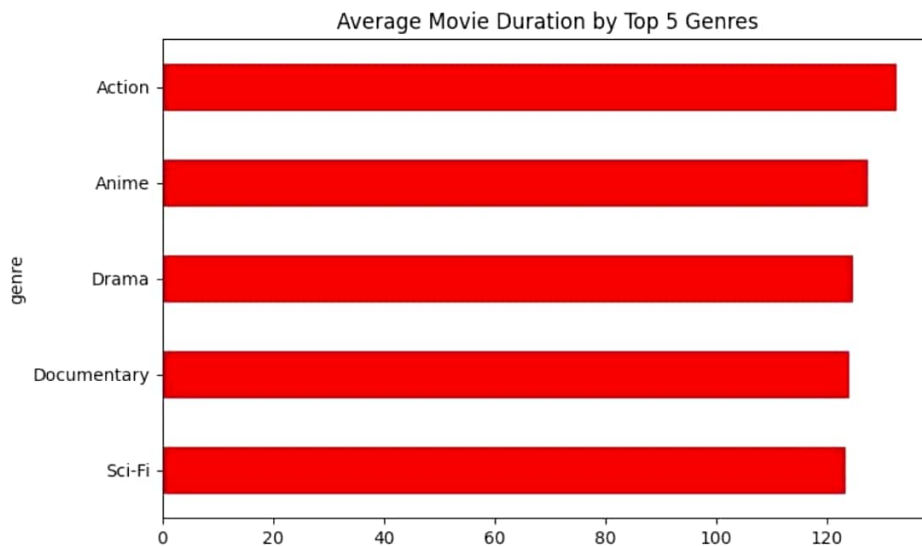


Fig 11.9: Average duration by genre

The five genres shown differ by at most a few minutes in average duration, too small a gap to describe any genre as systematically longer than another in this dataset, unlike real-world catalogues where Documentary and Drama titles are often measurably longer than Comedy or Kids titles.

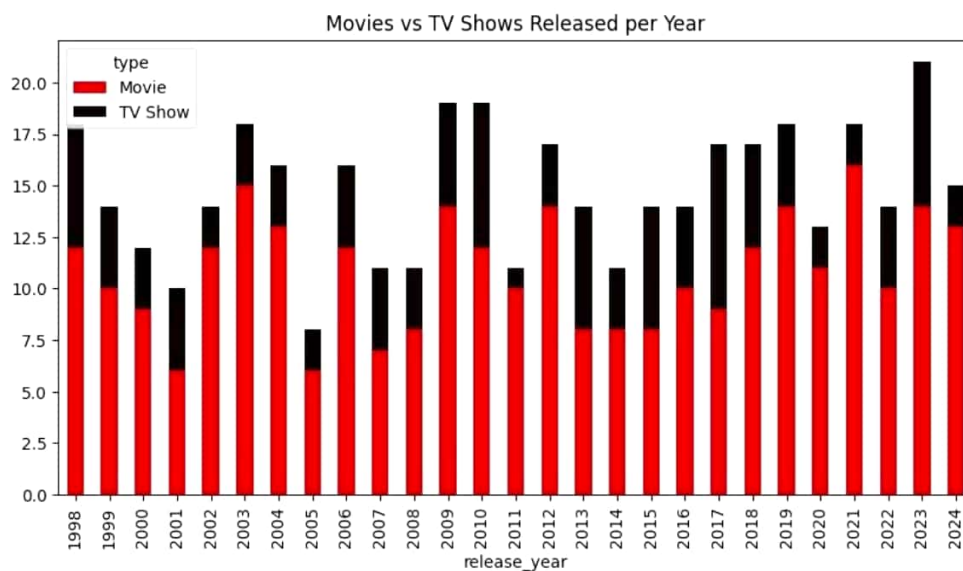


Fig 11.10: Movies vs TV Shows per year

The Movie segment of each bar is visibly larger than the TV Show segment in almost every year, confirming that the 73/27 split in Fig 11.1 holds consistently rather than being driven by one or two outlier years. No year shows TV Shows overtaking Movies, which would have been the first thing to check had the overall split looked less stable.

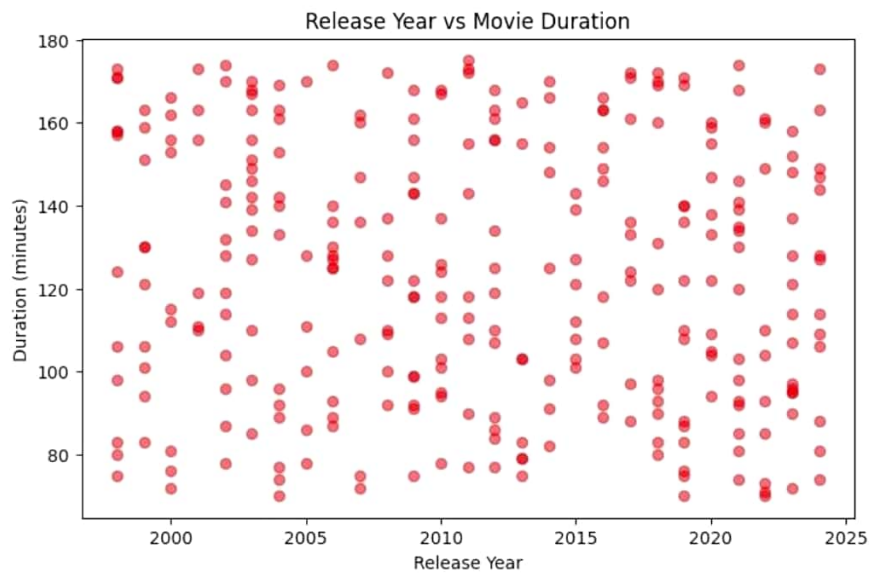


Fig 11.11: Release year vs duration (scatter)

The points fill the plot area evenly with no visible slope, indicating no relationship between a movie's release year and its runtime in this catalogue — recent movies are not systematically longer or shorter than older ones. A trend line fitted to this data would be expected to come out close to flat, reinforcing that duration and release year vary independently here.

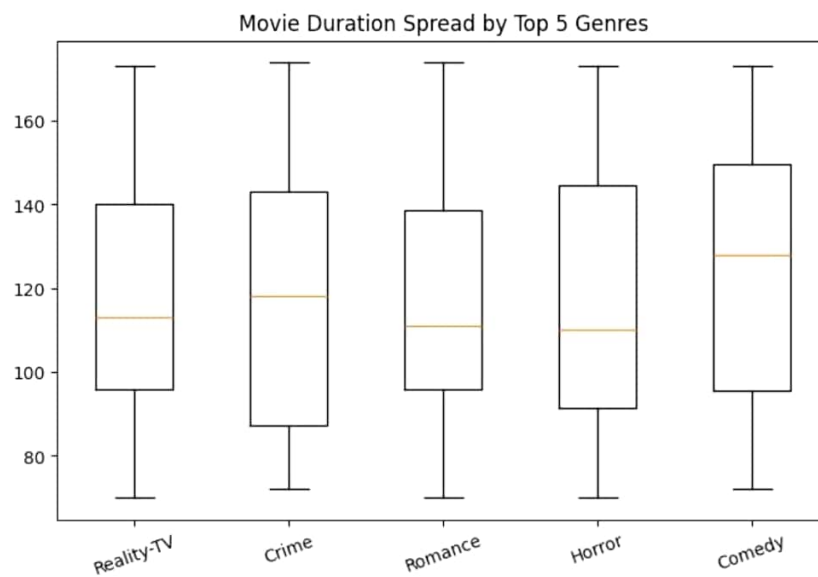


Fig 11.12: Duration spread by genre (box plot)

Median duration is within about 10 minutes across all five genres, though Comedy's box sits noticeably higher than Horror's, hinting at a mild tendency for comedies to run longer in this sample. None of the five genres shows a noticeably different spread (box height) either, so the similarity holds for both the typical value and the variability around it.

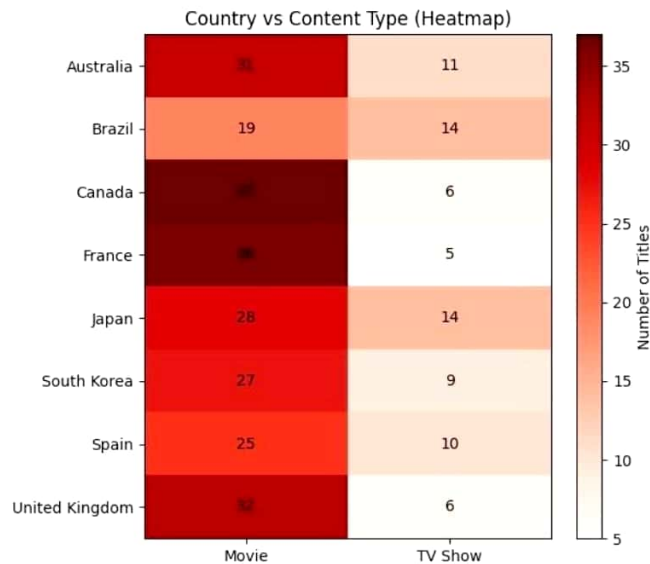


Fig 11.13: Country vs type (heatmap)

Canada and France stand out with Movie counts above 35 against single-digit TV Show counts, while Brazil and Japan are comparatively balanced, each with 14 TV Shows against 19–28 Movies — the clearest country-level contrast in the whole dataset. This is also the only chart in the report where two countries (Brazil and Japan) visibly break from the otherwise Movie-heavy pattern shown in Fig 11.10.

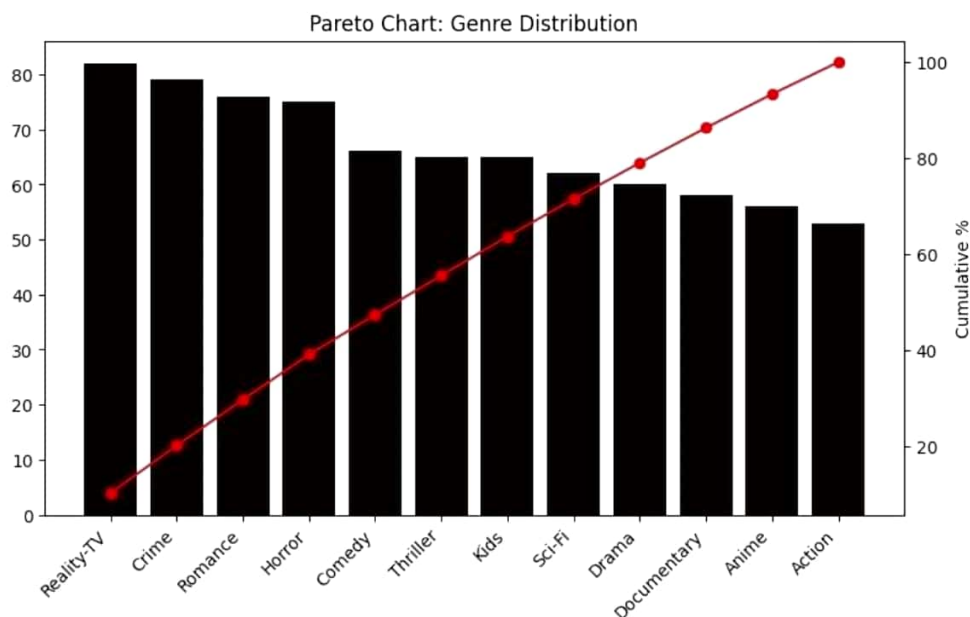


Fig 11.14: Pareto chart of genres

The cumulative-percentage line rises in roughly equal steps rather than levelling off after two or three bars, which is the Pareto chart's way of confirming numerically what Fig 11.3 suggested visually: no small set of genres dominates this catalogue. A classic 80/20 pattern would show the line reaching 80% within the first two or three genres; here it takes roughly eight of the eleven genres to cross that mark.

12. Advantages

- Running in Colab meant no group member had to install Python, Pandas or Matplotlib locally.
- Cell-by-cell execution made it straightforward to check each statistic before building the next one on top of it.
- Explicit handling of missing director/country values avoided silently undercounting genres or countries.
- The same notebook logic could be pointed at a different, similarly structured CSV with almost no changes.
- Fourteen chart types give a genuinely varied view of the data rather than repeating one chart style.
- Every result in this report can be traced back to the specific cell that produced it, which made writing Chapters 9 and 11 largely a matter of transcription rather than re-derivation.

13. Limitations

- Colab's session storage is temporary; the uploaded CSV and generated charts disappear when the runtime disconnects unless downloaded first.
- The analysis is descriptive only — it counts and compares, but does not predict or recommend.
- Genre and country statistics depend on consistent spelling and comma-separation in the source file.
- The dataset used here is a 400-row sample rather than a full Netflix export, so absolute counts should not be generalised.
- Re-running the notebook requires re-uploading the CSV each time, since Cell 1 does not check whether the file already exists in the session.

14. Applications

- A template notebook for students learning Pandas and Matplotlib on categorical data.
- A quick first pass over any content catalogue before building a more advanced analytics pipeline.
- A demonstration piece for showing cleaning, aggregation and multiple chart types in one place.
- Adaptable to similarly structured datasets, such as a music-streaming or e-commerce catalogue.

15. Future Scope

- Persist the uploaded dataset and generated charts to Google Drive so they survive a runtime reset.
- Add a recommendation component based on genre and description similarity.
- Parameterise the notebook to accept any CSV with the same column names via a Colab form.
- Compare this catalogue against a second dataset to identify overlaps and gaps.
- Wrap the cleaning and aggregation steps into reusable functions once the notebook's logic is finalised, so the same code can be imported into other projects instead of copy-pasted cell by cell.

16. Conclusion

Working entirely inside Google Colab, this project turned a 400-row Netflix-style CSV into fourteen charts and a text summary, after first resolving the one real data-quality issue in the file — 109 missing director values. Chapter 9 of this report shows that process cell by cell, with the exact output Colab produced at each step, which is the clearest way the group could demonstrate that the implementation works as described rather than only in principle.

The discussion in Chapter 11 goes further and shows that most of the patterns in this dataset are mild rather than dramatic — no single country, genre or rating dominates the catalogue — which is itself a useful, evidence-based conclusion rather than an assumed one. Taken together, the notebook, this report and the two appendices form a complete, self-contained record of the project: what was built, how it was run, what it produced, and what that output means.

17. References

- Pandas Documentation — <https://pandas.pydata.org/docs/>
- Matplotlib Documentation — <https://matplotlib.org/stable/contents.html>
- Google Colaboratory Documentation — <https://colab.research.google.com/>
- Python 3 Official Documentation — <https://docs.python.org/3/>

18. Appendix A: Consolidated Script

The 22 Colab cells reproduced individually in Chapter 9 are combined below into a single linear script, exactly as they would run outside Colab (with the `files.upload()` and `files.download()` calls replaced by plain file paths). This is provided purely as a convenient single-file reference and adds no logic beyond what Chapter 9 already shows.

```
import pandas as pd
import matplotlib.pyplot as plt

# Cell 3: Load and inspect
df = pd.read_csv("netflix_titles.csv")
print(df.shape)
print(df.head())

# Cell 4-5: Missing values and cleaning
print(df.isnull().sum())
df["director"] = df["director"].fillna("Not Specified")
df["country"] = df["country"].fillna("Unknown")
df.drop_duplicates(subset="show_id", inplace=True)

# Cell 6-7: Type distribution
type_counts = df["type"].value_counts()
plt.pie(type_counts, labels=type_counts.index, autopct="%1.1f%%")
plt.show()

# Cell 8: Top countries
top_countries = df["country"].value_counts().head(10)
top_countries.sort_values().plot(kind="barh"); plt.show()

# Cell 9: Top genres (explode multi-valued column)
genre_series = df["listed_in"].str.split(", ").explode()
top_genres = genre_series.value_counts().head(10)
top_genres.sort_values().plot(kind="barh"); plt.show()

# Cell 10: Rating distribution
df["rating"].value_counts().plot(kind="bar"); plt.show()

# Cell 11: Titles per release year
df["release_year"].value_counts().sort_index().plot(kind="line", marker="o");
plt.show()

# Cell 12: Movie duration statistics
movie_durations = df[df["type"]=="Movie"]["duration"].str.replace("
min", "", regex=False).astype(int)
print(movie_durations.mean(), movie_durations.min(), movie_durations.max())
plt.hist(movie_durations, bins=20); plt.show()

# Cell 13: Top directors
df[df["director"]!="Not Specified"]
["director"].value_counts().head(10).sort_values().plot(kind="barh"); plt.show()

# Cell 14: Titles added per year
df["year_added"] = df["date_added"].str.extract(r"(\d{4})").astype(int)
df["year_added"].value_counts().sort_index().plot(kind="bar"); plt.show()

# Cell 15: Average duration by genre
genre_duration_exp = df[df["type"]=="Movie"].assign(
    genre=df["listed_in"].str.split(", ").explode("genre")
)
genre_duration_exp.groupby("genre")["duration"].apply(
    lambda s: s.str.replace(" min", "", regex=False).astype(int).mean()
).sort_values(ascending=False).head(5)

# Cell 16: Movies vs TV Shows per year
pivot = df.pivot_table(index="release_year", columns="type", values="show_id",
aggfunc="count").fillna(0)
pivot.plot(kind="bar", stacked=True); plt.show()

# Cell 17-20: Scatter, box plot, heatmap, Pareto chart (see Chapter 9 for full code)

# Cell 21: Export summary report
with open("summary_report.txt", "w") as f:
    f.write(f"Total titles analysed : {len(df)}\n")
    f.write(type_counts.to_string())
```

19. Appendix B: Generated summary_report.txt

The exact contents of the file produced and downloaded by Cell 21 are reproduced below, unedited, as a second form of evidence that the pipeline runs end to end and that the numbers quoted throughout this report were taken directly from that output rather than computed separately for the write-up.

```
MOVIE / NETFLIX DATASET ANALYZER - SUMMARY REPORT
=====
```

```
Total titles analysed : 400
```

```
Content type distribution:
```

```
type
Movie      293
TV Show    107
```

```
Top 10 countries:
```

```
country
Canada      43
Australia   42
Japan       42
France      41
United Kingdom 38
South Korea 36
Spain       35
Brazil      33
India       32
United States 30
```

```
Top 10 genres:
```

```
listed_in
Reality-TV  82
Crime       79
Romance     76
Horror      75
Comedy      66
Thriller    65
Kids        65
Sci-Fi      62
Drama       60
Documentary 58
```

```
Average movie duration: 123.2 minutes
```